KOENIG

step forward



# Service Communication Patterns

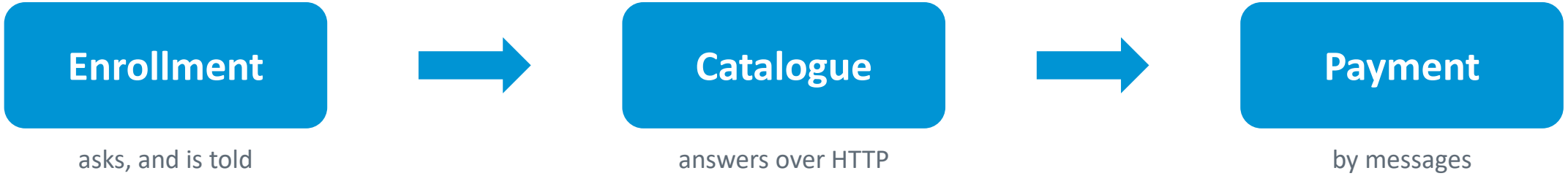
Calling services safely and talking through messages

# What You Will Learn

- Sync or async: choose per interaction
- Calling another service
- Timeouts
- Retry and the circuit breaker
- Messages between our services
- What if the broker is down?

# Demo Scenario: Learnly Services Must Talk

Catalogue knows the courses and Enrollment sells the seats, but today they ignore each other: Enrollment never asks whether a course exists, is open or what it costs, and it confirms without any payment. We make them talk to each others.




**What can go wrong**

- **Neighbour is slow**
- **Neighbour is down**
- **Message lost**

# Where We Left Things

| Service | Today |
|---|---|
| **Catalogue** | Owns the courses (Java, from the REST and JPA modules); we add a price |
| **Enrollment** | Stateless Kotlin service with seat batches; confirms an enrollment at once |
| **It should ask** | Catalogue: does the course exist, is it open, what does it cost? |
| **It should ask** | Payment: please charge this student, and tell me the result |
| **Now** | Make the services talk to each others |

**The services already exist; we only connect them**

# Two Ways to Talk

| | Synchronous: call and wait | Asynchronous: send and move on |
|---|---|---|
| **The caller** | waits for the answer | continues at once |
| **The neighbour** | must be up right now | may be down for a while |
| **When it fails** | the caller feels it at once | the message waits, nobody notices |
| **Learnly example** | Enrollment asks Catalogue | Enrollment asks Payment |
| **We need** | timeout, retry, circuit breaker | a way not to lose the message |

**Neither is better: the interaction decides**

# Calling Another Service

- **A port:** Enrollment's own question: CourseCatalogPort
- **A declarative client:** we describe the call, Spring writes it
- **An adapter:** the only class that knows the neighbour
- **An address:** in application.properties, never in code

```
@HttpExchange("/api/courses")
interface CatalogueClient {

    @GetExchange("/{id}")
    fun get(@PathVariable id: String): CatalogueCourse
}

# application.properties
spring.http.serviceclient.catalogue.base-url=
    http://localhost:8101
```

**No RestClient code, no JSON code: an interface and an address**

# The @HttpExchange Client

- **What:** a Java interface that describes an HTTP call; Spring writes the class
- **Annotations:** @HttpExchange (base path), @GetExchange (verb and path), @PathVariable
- **Register:** @ImportHttpServices(group, types) on a configuration class
- **Dependency:** spring-boot-starter-restclient
- **Address:** spring.http.serviceclient.<group>.base-url

```
@HttpExchange("/api/courses")
interface CatalogueClient {
    @GetExchange("/{id}")
    fun get(@PathVariable id: String): CatalogueCourse
}

@Configuration
@ImportHttpServices(group = "catalogue",
    types = [CatalogueClient::class])
class HttpClientsConfig

# application.properties
spring.http.serviceclient.catalogue.base-url=
    http://localhost:8101
```

**We describe the call; Spring writes the implementation at startup**

# Translate What the Neighbour Says

| The neighbour says | Enrollment says |
|---|---|
| **200 with the course** | the course, in our own words (title, price, open) |
| **404 not found** | 404 Course not found: an answer, not a failure |
| **400 or 409 (our own mistake)** | our own clear error, never retried |
| **500, slow, down** | 503 Catalogue unavailable |
| **Breaker open (no call made)** | 503 Catalogue paused |

**The adapter says UNAVAILABLE, never 503: the handler of the earlier module maps the kind**

**A 500 of the neighbour must never become a 500 of ours**

# Demo: A Call from Enrollment to Catalogue

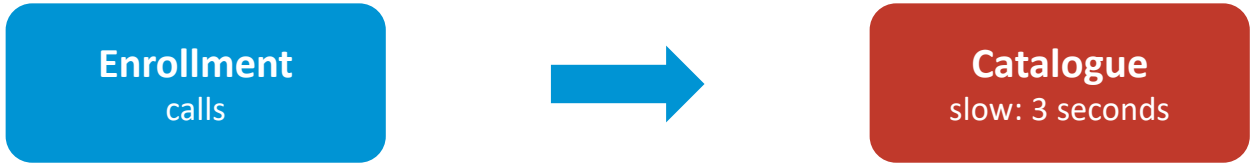
## What we do

- **Meet:** the prepared Catalogue and its contract.
- **Add:** a port, a client interface and an adapter.
- **Ask:** Catalogue before enrolling.
- **Break:** Catalogue on purpose.

## What it shows

- **Known course:** enrolled; the price comes from Catalogue.
- **Unknown course:** 404, the neighbour's answer kept.
- **Draft course:** 409, not open.
- **Catalogue failing:** a clean 503, never a 500.

# Never Wait Forever: Timeouts




**Without a timeout: wait 3 seconds, and so does the student**

**With a timeout of 1 second: give up, answer 503 at once**

**Two timeouts**
connect: how long to reach the neighbour
read: how long to wait for its answer

```
spring.http.clients.connect-timeout=1s
spring.http.clients.read-timeout=1s
```

**A slow neighbour is a failing neighbour that has not told you yet**

# The Timeout Properties

- **What:** two settings of the HTTP clients Spring creates for us
- **Comes from:** Spring Boot: no request-factory code
- **Where:** application.properties
- **Mind the name:** spring.http.clients, with an s
- **Check:** always prove a timeout with a slow neighbour

```
# application.properties
spring.http.clients.connect-timeout=1s
spring.http.clients.read-timeout=1s
```

**connect: how long to reach the neighbour**
read: how long to wait for its answer

**A timeout is a setting, not code**

# Retry: When and How

**Retry**
temporary and safe to repeat

**Do not retry**
a 404, our own mistake, a payment

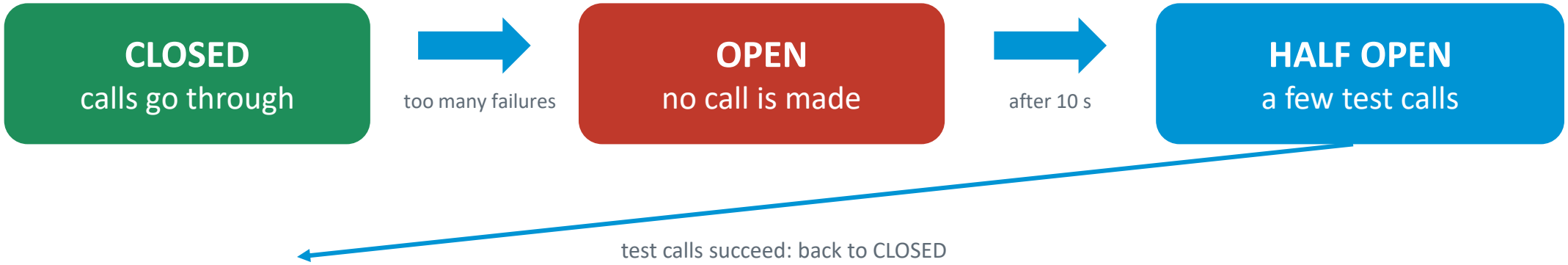
**Always**
a limit and a growing pause

```
resilience4j.retry.instances.catalogue
  .max-attempts=3
  .wait-duration=100ms
  .enable-exponential-backoff=true
  .ignore-exceptions=
     HttpClientErrorException,
     CallNotPermittedException
```

- **Attempt 1:** fails, wait 100 ms
- **Attempt 2:** fails, wait 200 ms
- **Attempt 3:** last one, then give up

**Retry what is safe and temporary; never retry forever**

# The Circuit Breaker




```
resilience4j.circuitbreaker.instances.catalogue
  .sliding-window-size=6                  # judge the last 6 calls
  .failure-rate-threshold=50              # open when half of them failed
  .wait-duration-in-open-state=10s        # stay open 10 seconds
```

**Stop calling a neighbour that keeps failing, and let it recover**

# Resilience4j

- **What:** a library of retry, circuit breaker and more, used with annotations and properties
- **resilience4j-spring-boot4:** the starter for Spring Boot 4: write its version by hand, it is not in the Resilience4j BOM
- **spring-boot-starter-aspectj:** so that the annotations work
- **spring-boot-starter-actuator:** to see the breaker at /actuator/circuitbreakers

```
// build.gradle.kts
implementation(
  "org.springframework.boot:" +
  "spring-boot-starter-actuator")
implementation(
  "org.springframework.boot:" +
  "spring-boot-starter-aspectj")
implementation(
  "io.github.resilience4j:" +
  "resilience4j-spring-boot4:2.4.0")
```

**Without :2.4.0 Gradle cannot find the starter**

**Three dependencies; the numbers live in application.properties**

# @Retry and @CircuitBreaker

```
@Retry(name = "catalogue",
       fallbackMethod = "unknownOrUnavailable")
@CircuitBreaker(name = "catalogue")
override fun find(id: String): CourseInfo? {
    ...
}

// the most specific fallback wins
private fun unknownOrUnavailable(
   id: String, e: HttpClientErrorException.NotFound)
   : CourseInfo? = null
```

```
resilience4j.retry.instances.catalogue
  .max-attempts=3
  .wait-duration=100ms
  .enable-exponential-backoff=true
  .ignore-exceptions=...404, ...open

resilience4j.circuitbreaker.instances.catalogue
  .sliding-window-size=6
  .minimum-number-of-calls=6
  .failure-rate-threshold=50
  .wait-duration-in-open-state=10s
```

**Trap: the fallback goes on @Retry (the outer one); on @CircuitBreaker it would hide the failure from the retry**

**Two annotations on one method, every number a property**

# Demo: Timeouts, Retry and Circuit Breaker

## What we do

- **Slow:** Catalogue takes 3 seconds: the student waits.
- **Add:** two timeout properties.
- **Add:** Resilience4j: two annotations, a few settings.
- **Break:** Catalogue: a hiccup, then really down.

## What it shows

- **Timeout:** 503 after about a second.
- **Hiccup:** retried, the student never notices.
- **Down:** breaker OPEN, answers in milliseconds.
- **Later:** after ten seconds it tests again.

# Messages Between Our Services

| payment.requests | payment.results |
| --- | --- |
| command | reply |
| Enrollment asks Payment | Payment tells Enrollment |

- **A command:** asks one receiver to do something
- **A reply:** tells the result, on its own queue
- **JMS and queues:** as in the JMS module: JmsTemplate to send, @JmsListener to receive

**Two queues carry the whole enrollment story**

# JmsTemplate and @JmsListener

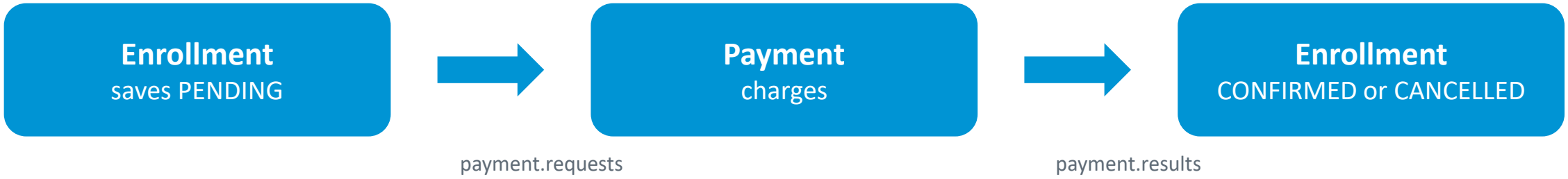
- **Dependency:** spring-boot-starter-artemis
- **Broker address:** spring.artemis.mode and broker-url
- **Send:** JmsTemplate.convertAndSend(queue, json text)
- **Receive:** @JmsListener(destination = queue)
- **One class:** knows the broker: a port and an adapter

```
// the only class that sends
@Component
class JmsPaymentRequests(
    jms: JmsTemplate, json: JsonMapper
) : PaymentRequestPort {
  override fun ask(...) = jms.convertAndSend(
      "payment.requests",
      json.writeValueAsString(message))
}

// a listener has no logic
@JmsListener(destination = "payment.results")
fun onResult(text: String) { ... service ... }
```

**As in the JMS module: the service never sees the broker**

# The Payment Flow: Ask, Then Be Told




**Enrollment does not wait: the answer to the student is 201 PENDING**

**Payment failed? Enrollment cancels and gives the seat back**

**For a moment the student is PENDING, and that is fine**

**No distributed transaction: each service decides alone and tells the others**

# Demo: The Payment Flow

## What we do

- **Code:** Payment live: a listener and a sender.
- **Teach:** Enrollment to start PENDING and to ask.
- **Run:** a good payment and a declined card.

## What it shows

- **Good payment:** 201 PENDING, then CONFIRMED.
- **Declined:** CANCELLED, the seat comes back.
- **Answer at once:** Enrollment never waits for Payment.

# Key Takeaways

1. **Two questions**
   Does the caller need the answer now? Is it safe to repeat?

2. **Protect calls**
   Timeout, retry for what is safe, circuit breaker.

3. **Translate**
   A 404 is an answer; a 500 of theirs is never a 500 of ours.

4. **Messages**
   For what can wait: ask by message, be told by message.

5. **When the broker is down**
   Write the message with the change: the outbox.

6. **Use libraries**
   @HttpExchange, Resilience4j, JmsTemplate: configured.



# Happy Learning :)